

INT201 Decision, Computation and Language

Lecture 2 – Deterministic Finite Automata

Dr Yushi Li 最基础的machine



Xi'an Jiaotong-Liverpool University

西交利物浦大學

Controlling a toll gate

收费站

When a car arrives at the toll gate, the gate is closed. The gate opens as soon as the driver has payed 25 cents. We assume that we have only three coin denominations: 5, 10, and 25 cents. Notably, no excess change is returned.

After having arrived at the toll gate, the driver inserts a sequence of coins into the machine. At any moment, the computer has to decide whether or not to open the gate, i.e., whether or not the driver has paid 25 cents (or more).



Controlling a toll gate

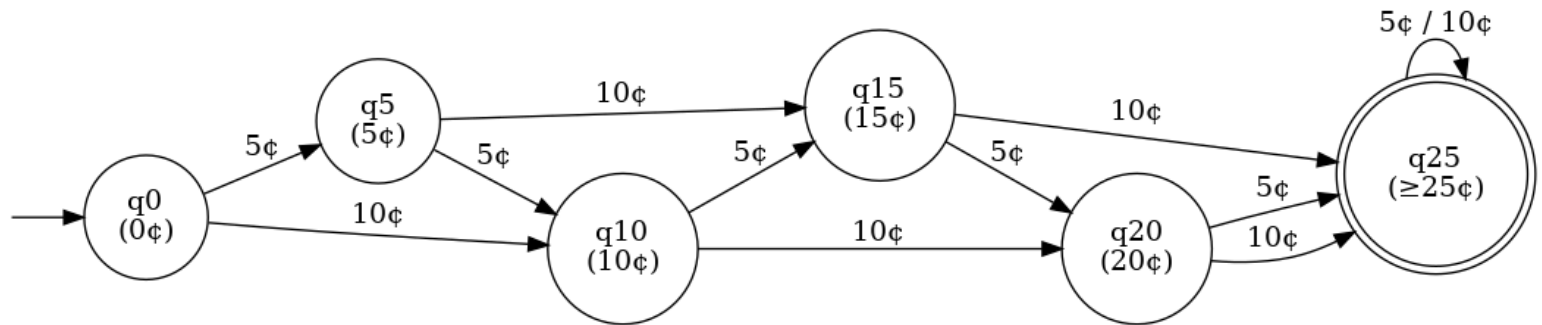
- The machine is in state q_0 , if it has not collected any money yet.
- The machine is in state q_1 , if it has collected exactly 5 cents.
- The machine is in state q_2 , if it has collected exactly 10 cents.
- The machine is in state q_3 , if it has collected exactly 15 cents.
- The machine is in state q_4 , if it has collected exactly 20 cents.
- The machine is in state q_5 , if it has collected 25 cents or more.



Controlling a toll gate

Question.

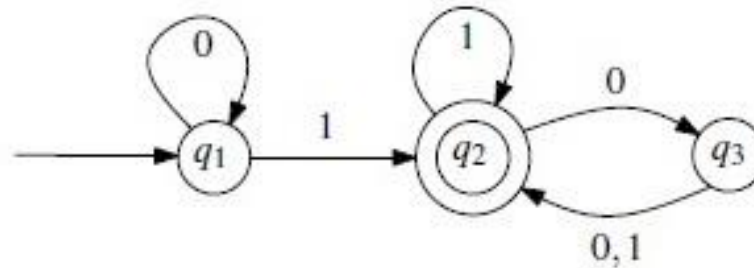
Can we draw a figure to represent the behavior of the computer for all possible sequences of coins?



Deterministic Finite Automata (DFA)

Example

We have a machine state diagram as below:



q_1 is the start state and q_2 is an accept state.

Question.

Can the input string 0101010 be accepted by this machine? 就是最后能到accept gate?

Which kind of strings can be accepted by this machine?



Deterministic Finite Automata (DFA)

Example

1. every string that end with one

2, every binary string having the property that there are an even number of 0s following the rightmost 1.



Deterministic Finite Automata (DFA)

确定性有限自动机 (Deterministic Finite Automaton, DFA) 是一种有限状态机 :

对给定输入串, 它按唯一确定的状态序列运行, 最后决定接受或拒绝该串。DFA 用来识别正则语言 (regular languages) 。

DFA is a finite-state machine that accepts or rejects a given string of symbols, by running through a state sequence uniquely determined by the string.

It is invented to **recognize a special class of formal language** and have many practical applications:

- Lexical analysis – scans the input program from beginning to end and divides it into tokens like identifiers, constants, and keywords and to remove comments and white space (specify tokens of programming languages).
- Model checking, reasoning about systems with objective of proving they satisfy useful properties.
- Statistical models for analyzing biological and textual sequences.



Deterministic Finite Automata (DFA)

We shall see that DFA can be used to describe

- Any finite set of strings
- Various infinite sets of strings, e.g.
 - ✓ strings having exactly 2 occurrences of the letter a
 - ✓ strings having more than 6 letters
 - ✓ strings in which letter b never comes before letter a



Deterministic Finite Automata (DFA)

limitations

DFA cannot be used to describe certain languages such as:

- the set of strings containing more a's than b's Text 不能去compare
- all words that remain the same if you read them back to front
- well-formed arithmetic expressions, if there is no limit on nesting of parentheses.

状态有限：DFA 只有有限多个状态，无法存储无限信息。

缺少记忆结构：不像栈自动机 (PDA) 有栈，或图灵机有磁带，DFA 不能对输入进行“计数”或“配对”超出有限界限。

只能处理“局部模式”：DFA 擅长检查局部固定的结构（如后缀、前缀、有限次计数），但无法进行“整体比较”或“无限嵌套”



Deterministic Finite Automata (DFA)

A DFA is defined as a 5-tuple:

$$M = (Q, \Sigma, \delta, q, F)$$

Where

Text

1. Q is a **finite** set of **states**,
2. Σ is a finite set of **symbols**, called the **alphabet** of the automaton, 有限字母表 (alphabet)
3. $\delta : Q \times \Sigma \rightarrow Q$ is a function, called the **transition function**,
4. $q \in Q$ is called the **initial state**,
5. $F \subseteq Q$ is a set of **accepting/terminal states**.

$\delta : Q \times \Sigma \rightarrow Q$: **转移函数** (对每个状态与输入符号都给出唯一的下一个状态)。

- 如果某些 (q, a) 在定义上没有结果, 通常把 DFA **完备化** (add a dead state) 使得 δ 在所有对上都有定义。

1. 形式定义 (五元组)

一个 DFA 定义为五元组

$$M = (Q, \Sigma, \delta, q_0, F)$$

- Q : 有限状态集合 (finite set of states) 。
- Σ : 有限字母表 (alphabet) 。
- $\delta : Q \times \Sigma \rightarrow Q$: **转移函数** (对每个状态与输入符号都给出唯一的下一个状态) 。
- 如果某些 (q, a) 在定义上没有结果, 通常把 DFA **完备化** (add a dead state) 使得 δ 在所有对上都有定义。
- $q_0 \in Q$: 初始状态 (start state) 。
- $F \subseteq Q$: 接受 (终态) 集合。

扩展转移函数 $\delta^* : Q \times \Sigma^* \rightarrow Q$ 定义为:

- $\delta^*(q, \epsilon) = q$ (空串) ,
- $\delta^*(q, xa) = \delta(\delta^*(q, x), a)$ 。

在处理字符串的时候就是一个字符一个的来



Deterministic Finite Automata (DFA)

State of a machine tells you something about the prefix that has been read so far. If the string is a member of the language of interest, the state reached when the whole string has been scanned will be an accepting state (a member of F).

Transition function δ tells you how state should change when an additional letter is read by the DFA.

2. 接受状态的意义

- 当输入串读完后, 如果最终停留在一个 **接受状态** (accepting state, 在集合 F 里), 就说明这个字符串符合我们要识别的语言; 否则就拒绝。
- 换句话说:

$$w \in L(M) \iff \delta^*(q_0, w) \in F$$

其中 δ^* 是扩展转移函数, 表示从初始状态 q_0 出发读完整个字符串 w 后到达的状态。

状态的含义

在 DFA 中, 每一个状态 (state) 其实都代表了“到目前为止 (prefix 已读部分) 我们已经知道的全部信息”。

也就是说, DFA 不需要保存完整的输入串, 而是通过“状态”来概括性地记录输入前缀的特征。

例如: DFA 用来判断“字符串中是否包含偶数个 a”。

只需要两个状态:

q_{even} (已经看到偶数个 a),

q_{odd} (已经看到奇数个 a)。

当前在哪个状态, 就能完整表示“前缀里 a 的数量是奇数还是偶数”



Deterministic Finite Automata (DFA)

Example of transition function

Initially the state is i and if the input word is $w = a_1 a_2 \dots a_n$ then, as each letter is read, the state changes and we get $q_1, q_2 \dots q_n$ defined by:

3. 转移函数 δ 的作用

- $\delta : Q \times \Sigma \rightarrow Q$

输入：当前状态 + 一个输入字符

输出：下一个状态。

- δ 的设计就是告诉 DFA：当再读一个字符时，机器应该如何更新“记忆”。

例子（继续“偶数个 a”的 DFA）：

- $\delta(q_{\text{even}}, a) = q_{\text{odd}}$
- $\delta(q_{\text{odd}}, a) = q_{\text{even}}$
- $\delta(q_{\text{even}}, b) = q_{\text{even}}$
- $\delta(q_{\text{odd}}, b) = q_{\text{odd}}$

也就是说：

- 如果当前状态表示“偶数个 a”，再读一个 a，就切换成“奇数个 a”；
- 如果当前状态表示“奇数个 a”，再读一个 a，就切换回“偶数个 a”；
- 读 b 时状态不变。

$$\begin{aligned} q_1 &= \delta(i, a_1) \\ q_2 &= \delta(q_1, a_2) \\ q_3 &= \delta(q_2, a_3) \\ &\vdots \\ q_n &= \delta(q_{n-1}, a_n) \end{aligned}$$



Deterministic Finite Automata (DFA)

2. 扩展转移函数的定义

设 DFA 的状态集是 Q , 字母表是 Σ 。

扩展转移函数:

$$\delta^* : Q \times \Sigma^* \rightarrow Q$$

定义规则:

1. 空串情况

$$\delta^*(q, \epsilon) = q, \quad \forall q \in Q$$

意思是: 如果输入什么都没有 (空串), 状态保持不变。

2. 归纳定义 (递归):

对于任意 $q \in Q$, 任意 $w \in \Sigma^*$, 任意 $a \in \Sigma$:

$$\delta^*(q, wa) = \delta(\delta^*(q, w), a)$$

- 先看前缀 w 输入后到达哪一个状态: $\delta^*(q, w)$ 。
- 然后再处理最后一个符号 a 。 (↓)

We can extend the definition of the transition function δ so that it tells us which state we reach after a word (not just a single letter) has been scanned:

In the above notation, extend the map $\delta : Q \times \Sigma \rightarrow Q$ to $\delta^* : Q \times \Sigma^* \rightarrow Q$ by defining:

$$\delta^*(q, \epsilon) = q \quad \text{for all } q \in Q$$

$$\delta^*(q, wa) = \delta(\delta^*(q, w), a) \quad \text{for all } q \in Q; w \in \Sigma^*; a \in \Sigma$$

你可以这样理解:

δ

*

δ

*

就是“把基本转移函数

δ

迭代应用在输入串的每个字符上”。

逐个读字符, 状态一步步更新, 最终得到结果状态。



Deterministic Finite Automata (DFA)

Example

$$\delta(q_0, a) = q_1 \text{ and } \delta(q_1, b) = q_2$$

Then

$$\delta^*(q_0, ab) = q_2$$

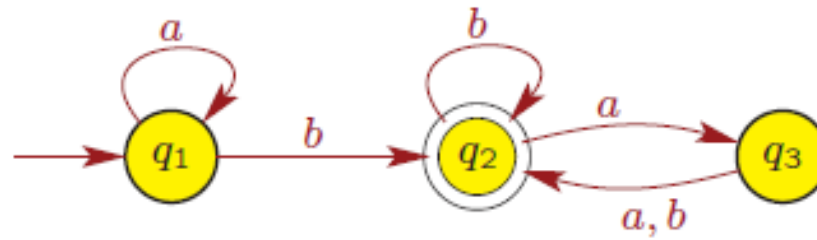
Proof Steps:

$$\left. \begin{array}{l} \delta^*(q_0, ab) = \delta(\delta^*(q_0, a), b) \\ \delta^*(q_0, a) = \delta(q_0, a) = q_1 \end{array} \right\} \delta^*(q_0, ab) = \delta(q_1, b) = q_2$$



Deterministic Finite Automata (DFA)

A DFA $M = (Q, \Sigma, \delta, q, F)$ is often depicted as a directed graph G_M (called **transition graph**) has exactly $|Q|$ vertices, each one labeled with a different $q_i \in Q$. For each transition function $\delta(q_i, a) = q_j$, the graph has edges (q_i, q_j) labeled (a) , (b) , and (a, b) , . The vertex associated with q_1 is called the initial vertex, while those labeled with q_f are the final vertices.



ε不能被接受 因为ε就会保持现在的状态



Deterministic Finite Automata (DFA)

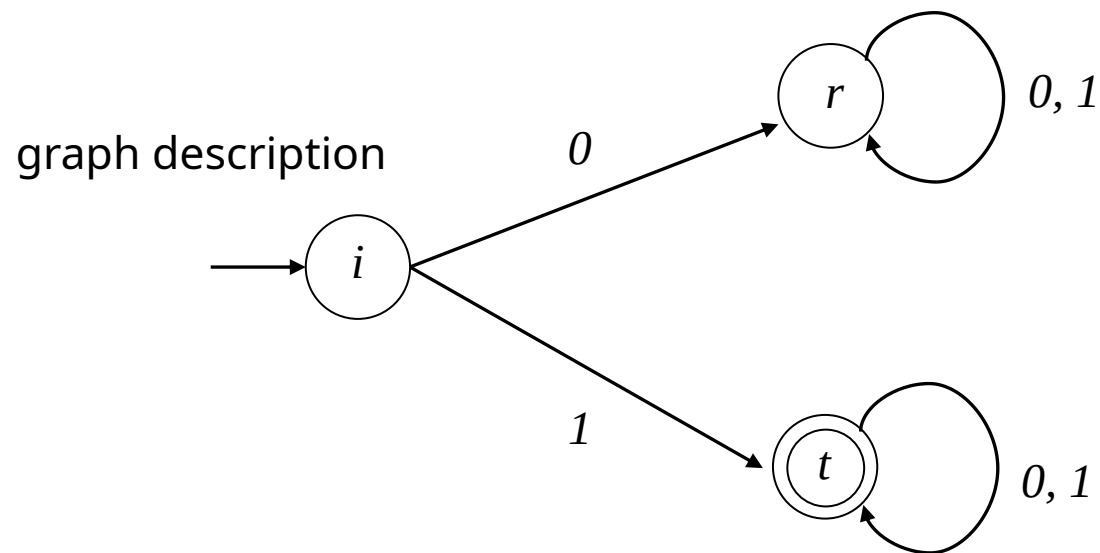
Let $M = (Q, \Sigma, \delta, q, F)$ be a finite automaton and let $w = w_0 w_1 \dots w_n$ be a string over Σ . Define the sequence $q_0, q_1, \dots, q_n$ of states, in the following way:

- $q_0 = q$,
 - $q_{i+1} = \delta(q_i, w_{i+1})$, for $i = 0, 1, \dots, n - 1$.
1. If $q_n \in F$, then we say that M accepts w .
 2. If $q_n \notin F$, then we say that M rejects w .



Deterministic Finite Automata (DFA)

Symbolic description of the example DFA



Deterministic Finite Automata (DFA)

Symbolic description of the example DFA

Automaton $M = (Q, \Sigma, \delta, q, F)$

Set of states $Q = \{i, t, r\}$, $\Sigma = \{0, 1\}$, $F = \{t\}$ and the transition function δ is given by

$$\delta(i, 0) = r \qquad \delta(i, 1) = t$$

$$\delta(t, 0) = t \qquad \delta(t, 1) = t$$

$$\delta(r, 0) = r \qquad \delta(r, 1) = r$$

It is simpler to describe a transition function by a table of values. In this example we have:

	0	1
i	r	t
t	t	t
r	r	r



Deterministic Finite Automata (DFA)

Symbolic description of the example DFA

If δ is a partial function (not defined for some state/letter pairs), then the DFA rejects an input if it ever encounters such a pair.

This convention often simplifies the definition of a DFA. In the previous example we could use transition table

1. 部分函数 δ 的含义

- 在标准 DFA 中, 转移函数 $\delta : Q \times \Sigma \rightarrow Q$ 必须在每个状态和每个输入符号上都有定义。
- 但有时为了简化定义, 可以使用 部分函数:

$$\delta : Q \times \Sigma \rightharpoonup Q$$

这里 “ $\rightharpoonup$ ” 表示不是每个输入对都定义结果。

- 解释:
 - 如果 DFA 在某个状态遇到一个未定义的输入符号, 它就 **立即拒绝**。
 - 不必为所有状态/符号组合都写转移表。

	0	1
i		t
t	t	t



Deterministic Finite Automata (DFA)

Exercise

DFA $M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_1\})$, where δ is given as:

$$\delta(q_0, 0) = q_0 \qquad \delta(q_0, 1) = q_1$$

$$\delta(q_1, 0) = q_0 \qquad \delta(q_1, 1) = q_2$$

$$\delta(q_2, 0) = q_2 \qquad \delta(q_2, 1) = q_1$$

Can the input word **110100** be accepted/recognized by this M ? no



Deterministic Finite Automata (DFA)

Exercise

How about the following infinite language. Can you give a DFA that accepts the words:

bad, baad, baaad, baaaad, ...?



Deterministic Finite Automata (DFA)

Exercise

Given the symbolic description of a DFA, can you draw its corresponding diagram?

add initial state Q_1 , accepted state Q_4

δ	a	b	c
Q_1	Q_2	Q_1	Q_3
Q_2	Q_4	Q_1	Q_2
Q_3	Q_2	Q_3	Q_1
Q_4	Q_4	Q_2	Q_3



important

Deterministic Finite Automata (DFA)

Language defined by DFA

Suppose we have a DFA M . A word $w \in \Sigma^*$ is said to be accepted or recognized by M if $\delta^*(q_0, w) \in F$, otherwise it is said to be rejected.¹ The set of all words accepted by M is called the language accepted by M and will be denoted by $L(M)$. Thus

$$2. L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}$$

3.

Any finite language is accepted by some DFA

A language A is called ⁴regular, if there exists a finite automaton M such that $A = L(M)$

check the language is regular? by



Deterministic Finite Automata (DFA)

Example

$A = \{w : w \text{ is a binary string containing an odd number of 1s}\}$

- The set of states is $Q = \{q_e, q_o\}$. If the finite automaton is in state q_e , then it has read an even number of 1s; if it is in state q_o , then it has read an odd number of 1s.
- The alphabet is $\Sigma = \{0, 1\}$.
- The start state is q_e , because at the start, the number of 1s read by the automaton is equal to 0, and 0 is even.
- The set F of accept states is $F = \{q_o\}$.
- The transition function δ is given by the following table:

	0	1
q_e	q_e	q_o
q_o	q_o	q_e



Deterministic Finite Automata (DFA)

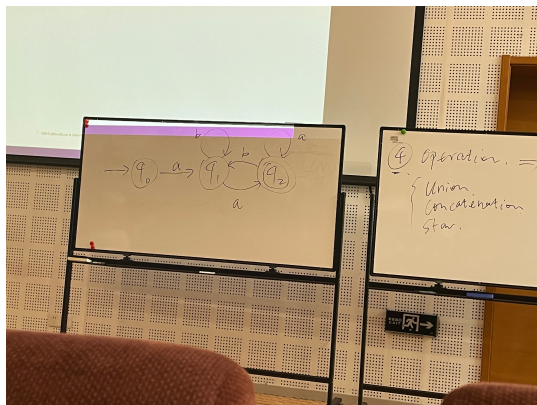
Exercise

Show that the language L is regular:

$$L = \{awa : w \in \{a,b\}^*\}$$

w can be any possible combination of a, b

Can you design a DFA that accepts this language?



Deterministic Finite Automata (DFA)

Regular operations on languages

Let **A** and **B** be two languages over the same alphabet.

The **union** of **A** and **B** is defined as:

$$\mathbf{A} \cup \mathbf{B} = \{w : w \in \mathbf{A} \text{ or } w \in \mathbf{B}\}$$

The **concatenation** of **A** and **B** is defined as:

$$\mathbf{AB} = \{ww' : w \in \mathbf{A} \text{ and } w' \in \mathbf{B}\}$$

The **star** of **A** is defined as:

$$\mathbf{A}^* = \{u_1 u_2 \dots u_k : k \geq 0 \text{ and } u_i \in \mathbf{A} \text{ for all } i = 1, 2, \dots, k\}$$



Deterministic Finite Automata (DFA)

Example of A^*

Given two languages $A = \{0, 01\}$ and $B = \{1, 10\}$. Then

$$A \cup B = \{0, 01, 1, 10\}$$

$$AB = \{01, 010, 011, 0110\}$$

$$A^* = \{\epsilon, 0, 01, 00, 001, 010, 0101, 000, 0001, 00101, \dots\}$$



Deterministic Finite Automata (DFA)

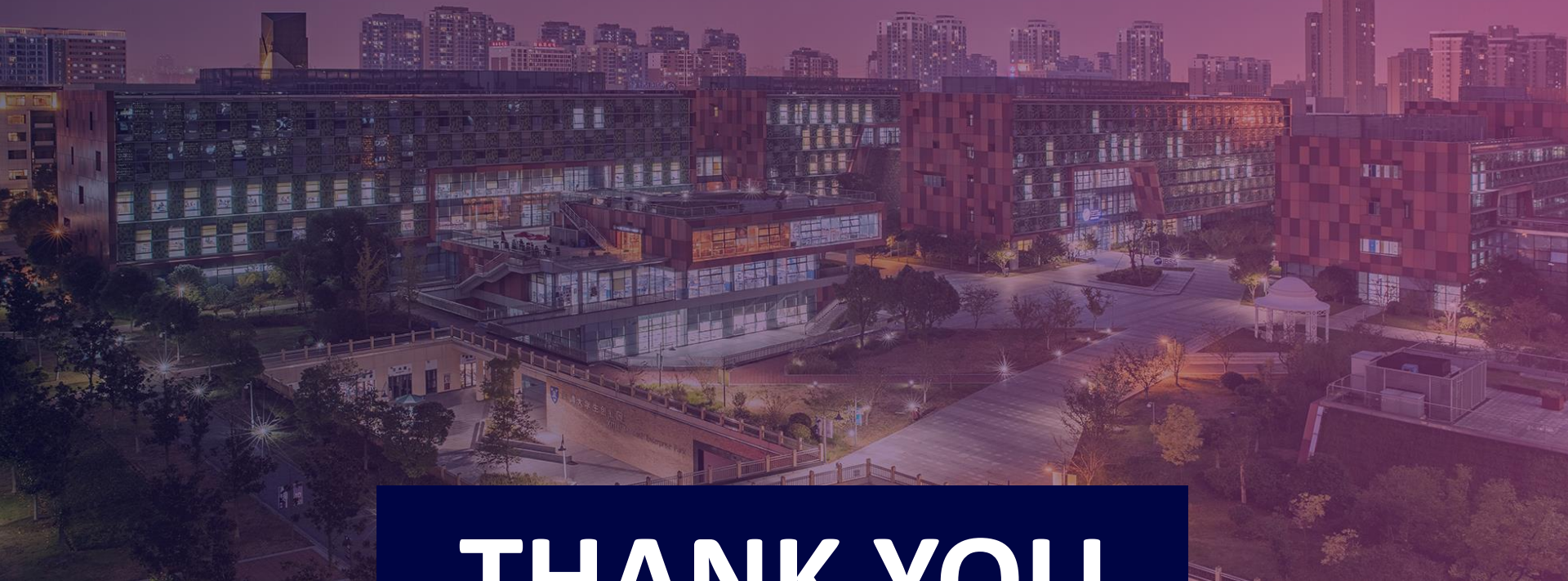
Theorem

The set of regular languages is closed under the union operation, i.e., if **A** and **B** are regular languages over the same alphabet Σ , then **A** $\cup$ **B** is also a regular language.

how to prove?

constructive method





THANK YOU



Xi'an Jiaotong-Liverpool University
西交利物浦大學

XJTLU | SCHOOL OF
FILM AND
TV ARTS